



API-Qualität: Validierung und Dokumentation

Ergänzende Folien zum Mini-Shop Beispielprojekt.

Request-Validierung

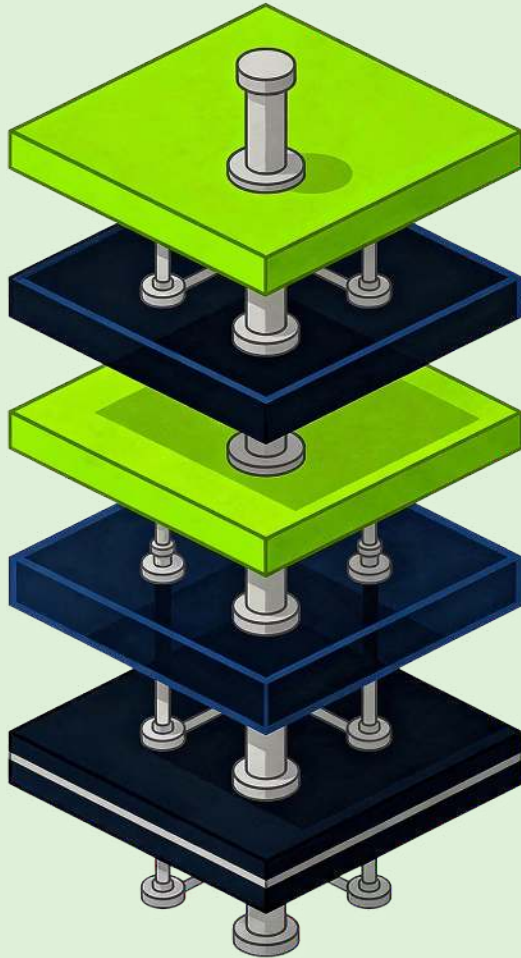
Mit Zod

API-Verträge

Mit OpenAPI

Interaktive Dokumentation

Mit Swagger UI



Warum diese Erweiterung?

Bisherige Iterationen

- Services
- Container
- Orchestrierung
- CI/CD

Jetzt: API-Qualität

- ☐ Welche Daten akzeptiert ein Service?
- ☐ Welche Fehler kann ein Client erwarten?
- ☐ Wie wird ein API-Vertrag sichtbar?

Problem ohne Validierung

Express nimmt JSON entgegen, aber prüft es nicht automatisch.

Problematische Requests

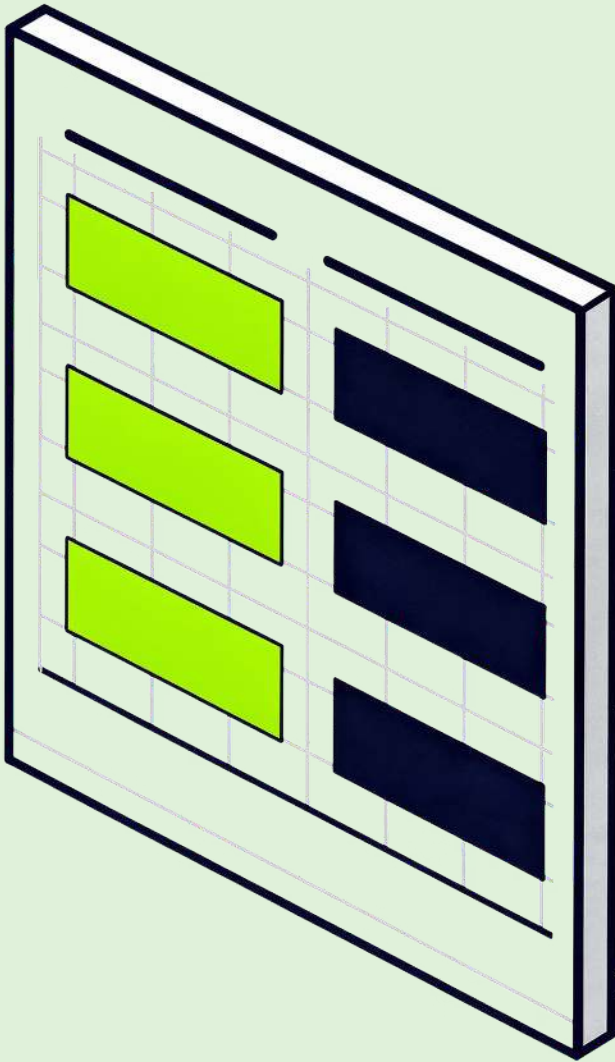
```
{}
```

```
{ "productId": "1", "quantity": 0 }
```

⊗ Ohne klare Validierung entstehen uneinheitliche Fehler oder falsche Annahmen im Code.

Fehlende Validierung führt zu:

- Undefinierten Fehlerzuständen
- Falschen Annahmen in der Fachlogik
- Schwer debuggbaren Problemen



Iteration 7: Zod

1

Request-Body validieren

POST /orders wird mit Zod geprüft

2

Fehler früh zurückgeben

Eindeutige Fehlermeldungen an der API-Grenze

3

API robuster machen

Verhalten wird vorhersehbar und verlässlich



Betroffener Service: **order-service**

Zod-Schema

Der `order-service` definiert ein Schema, das die erwartete Struktur zur Laufzeit beschreibt:

```
const createOrderSchema = z.object({  
  productId: z.string().regex(/^\\d+$/),  
  quantity: z.number().int().positive()  
});
```

productId

- Typ: String
- Muster: nur Ziffern (`/^\\d+$/`)

quantity

- Typ: Ganzzahl
- Bedingung: positiv (`> 0`)

Validierung im Endpoint

Vor der Fachlogik wird der Request geprüft. Erst danach werden Product- und Inventory-Service aufgerufen.

```
const result = createOrderSchema.safeParse(req.body);
if (!result.success) {
  return res.status(400).json({
    error: "Invalid request body",
    issues: result.error.issues
  });
}
```



Die Trennung von Validierung und Fachlogik macht den Code klarer und sicherer.

Gültiger Request

curl-Befehl

```
curl -X POST http://localhost:3000/orders \  
-H "Content-Type: application/json" \  
-d '{"productId":1,"quantity":2}'
```

Erwartete Antwort

201

Created

Der Request wird vom Gateway an den order-service weitergeleitet.

Ungültiger Request

curl-Befehl

```
curl -X POST http://localhost:3000/orders \  
-H "Content-Type: application/json" \  
-d '{"productId":"1","quantity":0}'
```

⚠ **quantity: 0** verletzt die Bedingung `positive()` im Zod-Schema.

Erwartete Antwort

400

Bad Request

Der Fehler entsteht **vor** den Downstream-Aufrufen – kein unnötiger Traffic zu Product- oder Inventory-Service.

Zod: Lernpunkte

JSON-Parsing ≠ Validierung

Ein geparster JSON-Body kann trotzdem ungültige Werte enthalten.

Validierung an der API-Grenze

Fehler sollten so früh wie möglich abgefangen werden.

Maschinenlesbare Fehler

Fehlerantworten sollten strukturiert und programmatisch auswertbar sein.

Einfachere Fachlogik

Wenn Eingaben vorher geprüft sind, wird die Geschäftslogik klarer und robuster.



Zod: Übungsaufgaben



Optionales Feld ergänzen

Ergänze ein optionales Feld `customerEmail` im Schema.



E-Mail validieren

Validiere das Feld als gültige E-Mail-Adresse mit Zod.



Route-Parameter validieren

Validiere Route-Parameter in weiteren Services.



Mit curl testen

Teste ungültige Requests mit curl und beobachte die Fehlerantworten.



Bonus: notes-Feld

Ergänze `notes` mit maximal 200 Zeichen.



Problem ohne API-Dokumentation

Clients müssen wissen, wie sie mit einem Service kommunizieren können – doch wenn das nur im Code steht, ist der API-Vertrag schwer sichtbar.



Welche Endpoints gibt es?



Welche Request-Bodies sind erlaubt?



Welche Responses kommen zurück?



Welche Fehlerfälle sind möglich?

Iteration 8: Swagger / OpenAPI

API-Vertrag beschreiben

Explizite, standardisierte
Beschreibung aller Endpoints

Vollständig dokumentieren

Endpoints, Schemas und Fehlerfälle
werden sichtbar

Im Browser ausprobieren

API direkt über Swagger UI testen

 Betroffener Service: **api-gateway**



Neue Gateway-Endpoints

Zwei neue Routen

GET /openapi.json

Liefert die vollständige OpenAPI-Spezifikation als JSON-Dokument.

GET /docs

Zeigt die interaktive Swagger UI im Browser an.

Verwendung

Die Spezifikation unter `/openapi.json` kann von Tools, CI-Pipelines und anderen Services konsumiert werden.

Die UI unter `/docs` richtet sich an Entwickler und Teams, die die API erkunden möchten.

OpenAPI beschreibt den Vertrag

Beispiel für den Request-Body `CreateOrderRequest`:

`CreateOrderRequest`:

`type: object`

`required:`

- `productId`
- `quantity`

`properties:`

`productId:`

`type: string`

`pattern: ^\d+$`

`quantity:`

`type: integer`

`minimum: 1`

❏ Das OpenAPI-Schema spiegelt exakt die Regeln des Zod-Schemas wider – beide müssen fachlich übereinstimmen.

- `required`: Pflichtfelder explizit deklariert
- `pattern`: Regex-Validierung für `productId`
- `minimum: 1`: Entspricht `positive()` in Zod

Swagger UI

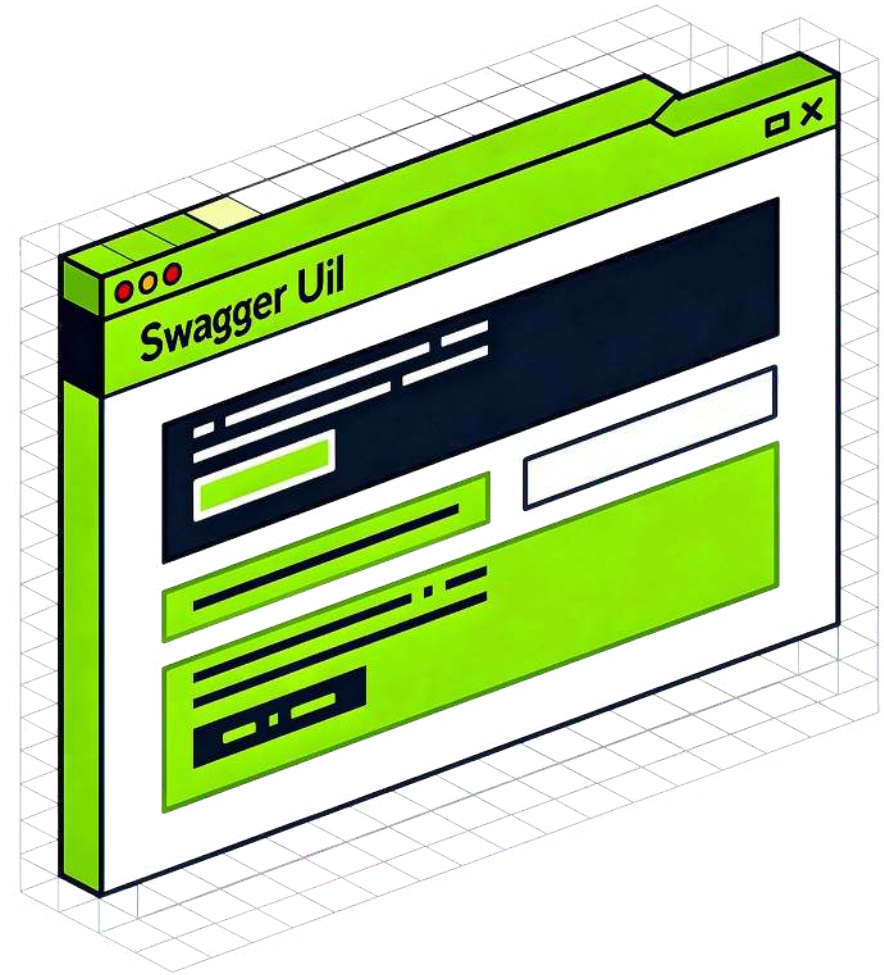
Starten

```
docker compose \  
-f compose/docker-compose.yml \  
up --build
```

Browser öffnen

```
http://localhost:3000/docs
```

✓ Die API kann dort gelesen und direkt ausprobiert werden – ohne zusätzliche Tools.



Dokumentation und Validierung

Zod – Laufzeit

Prüft Requests zur Laufzeit und gibt strukturierte Fehler zurück.



OpenAPI – Vertrag

Beschreibt den Vertrag für Clients und andere Teams.



Konsistenz – Pflicht

Beide müssen fachlich dasselbe ausdrücken.

⊗ Wenn Zod und OpenAPI auseinanderlaufen, entsteht ein **falscher Vertrag** – Clients erhalten falsche Erwartungen.

Swagger: Übungsaufgaben

1

Schema vergleichen

Vergleiche `CreateOrderRequest` mit dem Zod-Schema auf Übereinstimmung.

2

400 dokumentieren

Dokumentiere ein Beispiel für `400 Bad Request` in OpenAPI.

3

409 dokumentieren

Dokumentiere `409 Not enough inventory` als Fehlerfall.

4

Request ausführen

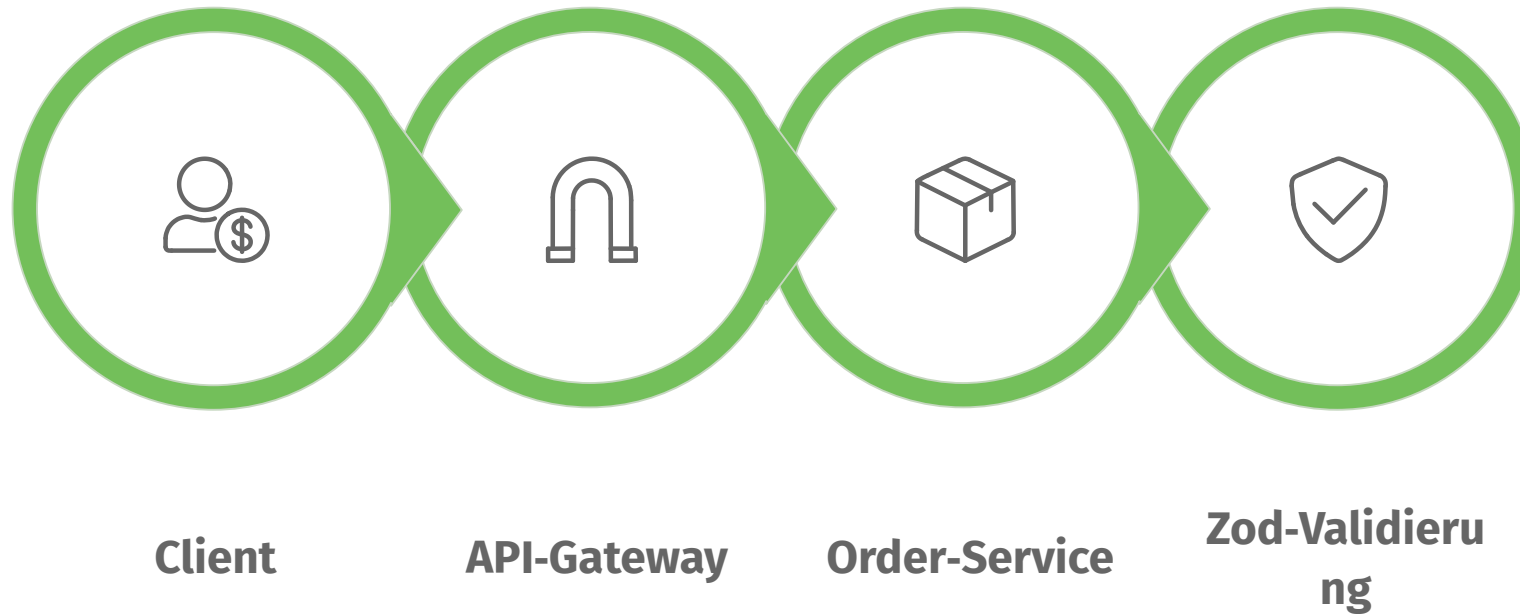
Führe einen Request direkt in der Swagger UI aus.

5

Bonus: GET `/products/{id}`

Ergänze den Endpoint im Gateway und in OpenAPI.

Zusammenspiel



Swagger macht den Vertrag sichtbar – Clients und Teams verstehen die API ohne den Code zu lesen.

Zod setzt den Vertrag zur Laufzeit durch – ungültige Requests werden abgewiesen, bevor Schaden entsteht.

Vergleich der Werkzeuge

Werkzeug	Zweck
Zod	Eingaben zur Laufzeit validieren
OpenAPI	API-Vertrag standardisiert beschreiben
Swagger UI	API-Dokumentation anzeigen und testen

Typische Fehlerbilder

OpenAPI erlaubt mehr als Zod

Clients senden Daten, die laut Dokumentation gültig sind – aber zur Laufzeit abgelehnt werden.

Zod erlaubt mehr als OpenAPI

Die Implementierung akzeptiert Daten, die laut Vertrag nicht erlaubt sein sollten.

Fehlerantworten nicht dokumentiert

Clients wissen nicht, welche Fehlercodes sie behandeln müssen.

Swagger-Beispiele funktionieren nicht

Beispiele in der Dokumentation sind veraltet oder falsch – Vertrauen geht verloren.

Unterschiedliche Annahmen

Gateway und Downstream-Service haben abweichende Erwartungen an die Datenstruktur.

Fazit

Microservices brauchen klare API-Grenzen – ohne sie entstehen Missverständnisse, Fehler und schwer wartbarer Code.

Zod

Hilft **innerhalb des Services** –
Eingaben werden zur Laufzeit
geprüft, Fachlogik bleibt sauber.

OpenAPI

Hilft **an der Schnittstelle** – der
Vertrag ist explizit, standardisiert
und maschinenlesbar.

Swagger UI

Hilft **Teams und Clients** – die API kann erkundet und direkt getestet werden.

- ✓ Zusammen machen sie API-Verhalten **sichtbarer, testbarer** und **verlässlicher**.

